WEEK 1 · LESSON 3 OF 24 · HANDOUT

Next-Token Prediction and Sampling

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=wjZofJX0v4M — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: The spelled-out intro to language modeling — building makemore — https://www.youtube.com/watch?v=PaCmpygFfXo

🎓 **Go deeper:** CS50 Seminar: How to Predict the Future with AI, One Word at a Time — https://www.youtube.com/watch?v=NGEEN0BzH-s

Your goal in this lesson: Turn model scores into probabilities, and implement greedy decoding, temperature, top-k and top-p sampling to see how they change an LLM's answers.

From scores to choices

At every step an LLM outputs a **score (logit)** for each token in its vocabulary. **Softmax** turns the scores into probabilities. Then a **decoding strategy** picks one token, it is added to the text, and the whole thing repeats.

Let's use a toy example: the model has just read "The price of beans in Kigali is" and gives these scores:

```
import numpy as np

tokens = ["low", "high", "good", "fair", "rising", "purple"]
logits = np.array([3.0, 2.5, 1.5, 1.0, 0.5, -2.0])

def softmax(z):
    e = np.exp(z - z.max())
    return e / e.sum()

for t, p in zip(tokens, softmax(logits)):
    print(f"{t:7s} {p:.3f}")
```

"low" gets about **0.49**, "high" about 0.30, and the silly "purple" almost nothing (0.003).

Strategy 1 — Greedy: always take the top token

```
print("greedy:", tokens[int(np.argmax(logits))])
```

Greedy decoding is **deterministic** (same answer every time) and good for short factual answers, but over long texts it tends to repeat itself and sound flat.

Strategy 2 — Sampling with temperature

Sampling picks a token **at random, in proportion to its probability**. **Temperature** reshapes the probabilities first by dividing the logits:

- temperature **below 1** → sharper: the top token dominates (more focused, predictable).

- temperature **above 1** → flatter: unlikely tokens get more chances (more varied, riskier).

```
for T in [0.2, 0.7, 1.0, 1.5]:
    p = softmax(logits / T)
    print(f"T={T}: " + ", ".join(f"{t} {q:.2f}" for t, q in zip(tokens, p)))
```

At T = 0.2 "low" gets about 92% of the probability; at T = 1.5 it drops to about 39% and even "purple" rises to about 1%.

```
rng = np.random.default_rng(0)
for T in [0.2, 1.0, 1.5]:
    picks = rng.choice(tokens, size=12, p=softmax(logits / T))
    print(f"T={T}:", " ".join(picks))
```

Strategy 3 — Top-k: only the k best candidates

```
def top_k_probs(logits, k):
    cut = np.sort(logits)[-k]
    kept = np.where(logits >= cut, logits, -np.inf)
    return softmax(kept)

print("top-3:", dict(zip(tokens, top_k_probs(logits, 3).round(2))))
```

Only "low", "high" and "good" can be chosen; nonsense like "purple" is impossible.

Strategy 4 — Top-p (nucleus): the smallest set that covers p

Top-p keeps the most likely tokens until their probabilities add up to p (for example 0.9), then samples among them. It adapts: when the model is sure, few tokens are kept; when it is unsure, more are.

```
def top_p_probs(logits, p_keep):
    probs = softmax(logits)
    order = np.argsort(probs)[::-1]
    cumulative = np.cumsum(probs[order])
    keep = order[: int(np.searchsorted(cumulative, p_keep)) + 1]
    out = np.zeros_like(probs)
    out[keep] = probs[keep]
    return out / out.sum()

print("top-p 0.9:", dict(zip(tokens, top_p_probs(logits, 0.9).round(2))))
```

With p = 0.9 the four best tokens are kept and "rising" and "purple" are dropped.

What this means when you use an LLM

Setting	Behaviour	Good for
Low temperature / greedy	Focused, repeatable	Extraction, classification, code, facts
Medium (about 0.7)	Balanced	Most chat and writing
High temperature	Creative, surprising, error-prone	Brainstorming, stories

- The same prompt can give **different answers** because of sampling. For tests and automation, keep randomness low or fix a seed where the tool allows it.

- Some newest models and APIs manage sampling for you and no longer expose these settings; then you steer them with the **prompt** instead (Week 2).
- Sampling explains part of **hallucination**: the model always produces **some** fluent next token, even when it has no reliable knowledge.

Key terms

- **Logit** — the raw score for a token.
- **Greedy decoding** — always pick the most likely token.
- **Temperature** — divides logits to make the distribution sharper (low) or flatter (high).
- **Top-k** — sample only among the k most likely tokens.
- **Top-p (nucleus)** — sample among the smallest set whose probabilities reach p.

Remember this: An LLM's answer depends not only on its probabilities but on how we sample from them; low randomness is focused and repeatable, high randomness is creative and riskier.

Name: Date:

Activity

Task 1

With T = 0.01, sample 20 tokens. What happens, and why does it match greedy decoding?

Task 2

Change the logits so the model is very sure (for example [8, 2, 1, 1, 0, −2]). How many tokens does top-p 0.9 keep now?

Task 3

Combine top-k = 3 with temperature 1.5. Which tokens can appear?

Task 4

Ask a chatbot the same creative question three times. How different are the answers?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What turns a model's logits into probabilities?
   - A. Softmax
   - B. Temperature
   - C. Tokenization
   - D. Sorting
2. What does greedy decoding do?
   - A. Always picks the most likely token
   - B. Picks a random token
   - C. Skips tokens
   - D. Picks the longest token
3. What does a low temperature (such as 0.2) do?
   - A. Speeds up the GPU
   - B. Makes the top token dominate, so output is more focused
   - C. Removes tokens
   - D. Makes choices more random
4. What does top-k = 3 mean?
   - A. Generate 3 tokens
   - B. Use 3 models
   - C. Sample only among the 3 most likely tokens
   - D. Repeat 3 times
5. How is top-p different from top-k?
   - A. It ignores probabilities
   - B. It keeps the smallest set of tokens whose probabilities add up to p
   - C. It only works for images
   - D. It keeps a fixed number of tokens
6. Which setting suits extracting a phone number from a message?
   - A. High temperature
   - B. Random
   - C. Low temperature or greedy
   - D. Top-k = 1000

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.